

Practical Lab Manual

High Performance Computing (410250)

For Final Year Computer Engineering (2019 Course)



Student Name: _____

Seat No / Roll No: _____ Class: _____

Branch: _____



Department of Computer Engineering

Al Jamia Mohammediyah Education Society's

Maulana Mukhtar Ahmad Nadvi Technical Campus

Mansoor Campus, Malegaon (Nashik)

Certificate

This is to certify that.....

Roll No:Exam Seat No:..... Class:

Branchhas Successfully Completed

High Performance computing Lab Experiments and Assignments as per the term work of
Savitri-bai Phule Pune University, Pune.

Subject Teacher

Head of Department

Principal

Maulana Mukhtar Ahmad Nadvi Technical Campus
Department of Computer Engineering

Vision of the Institute

Empowering society through quality education and research for the socio-economic development of the region.

Mission of the Institute

- Inspire students to achieve excellence in science and engineering.
- Commit to making quality education accessible and affordable to serve society.
- Provide transformative, holistic, and value-based immersive learning experiences for students.
- Transform into an institution of global standards that contributes to nation-building.
- Develop sustainable, cost-effective solutions through innovation and research.
- Promote quality education in rural areas.

Vision of the Department

To build strong research and learning environment producing globally competent professionals and innovators who will contribute to the betterment of the society.

Mission of the Department

- To create and sustain an academic environment conducive to the highest level of research and teaching.
- To provide state-of-the-art laboratories which will be up to date with the new developments in the area of computer engineering.
- To organize competitive event, industry interactions and global collaborations in view of providing a nurturing environment for students to prepare for a successful career and the ability to tackle lifelong challenges in global industrial needs.
- To educate students to be socially and ethically responsible citizens in view of national and global development.

Program Outcomes (POs)

Learners are expected to know and be able to

PO1	Engineering knowledge	Apply the knowledge of mathematics, science, Engineering fundamentals, and an Engineering specialization to the solution of complex Engineering problems.
PO2	Problem analysis	Identify, formulate, review research literature and analyze complex Engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and Engineering sciences.
PO3	Design / Development of Solutions	Design solutions for complex Engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and Environmental considerations.
PO4	Conduct Investigations of Complex Problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool Usage	Create, select, and apply appropriate techniques, resources, and modern Engineering and IT tools including prediction and modeling to complex Engineering activities with an understanding of the limitations.
PO6	The Engineer and Society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and Sustainability	Understand the impact of the professional Engineering solutions in societal and Environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of Engineering practice.
PO9	Individual and Team Work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
P10	Communication Skills	Communicate effectively on complex Engineering activities with the Engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions
P11	Project Management and Finance	Demonstrate knowledge and understanding of Engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary Environments.
P12	Life-long Learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO)

A graduate of the Computer Engineering Program will demonstrate

PSO1

Professional Skills-The ability to understand, analyze and develop computer programs in the areas related to algorithms, system software, multimedia, web design, big data analytics, and networking for efficient design of computer-based systems of varying complexities.

PSO2

Problem-Solving Skills- The ability to apply standard practices and strategies in software project development using open-ended programming environments to deliver a quality product for business success.

PSO3

Successful Career and Entrepreneurship- The ability to employ modern computer languages, environments and platforms in creating innovative career paths to be an entrepreneur and to have a zest for higher studies

Savitri Bai Phule Pune University
Final Year of Computer Engineering (2019 course)
410250: Laboratory Practice V (High Performance Computing)

Course Objectives:

- To understand and implement searching and sorting algorithms.
- To learn the fundamentals of GPU Computing in the CUDA environment.

Course Outcomes:

On completion of this course, the students will be able to

CO1: Analyze and measure performance of sequential and parallel algorithms.

CO2: Design and Implement solutions for multicore/Distributed/parallel environment

410250 : High Performance Computing	
Any 4 Assignments and 1 Mini Project are Mandatory	
Group 1	
1.	Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS .
2.	Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.
3.	Implement Min, Max, Sum and Average operations using Parallel Reduction.
4.	Write a CUDA Program for : 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C
5.	Implement HPC application for AI/ML domain.
Group 2	
6.	Mini Project: Evaluate performance enhancement of parallel Quicksort Algorithm using MPI
7.	Mini Project: Implement Huffman Encoding on GPU
8.	Mini Project: Implement Parallelization of Database Query optimization
9.	Mini Project: Implement Non-Serial Polyadic Dynamic Programming with GPU Parallelization

INDEX

Sr .No	Name Of The Experiment	Date Of Start	Date Of Completion	Marks	Sign
1	Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS.				
2	Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.				
3	Implement Min, Max, Sum and Average operations using Parallel Reduction.				
4	Write a CUDA program for : 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C				
5	Mini Project				

Experiment - 01

Aim:

Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. For

1. Use a Tree
2. An undirected graph for BFS and DFS.

Objective:

Student will learn:

1. The Basic Concepts of DFS, BFS.
2. Multiple Compiler Directives, library routines, environment variables available for OpenMP.

Theory:

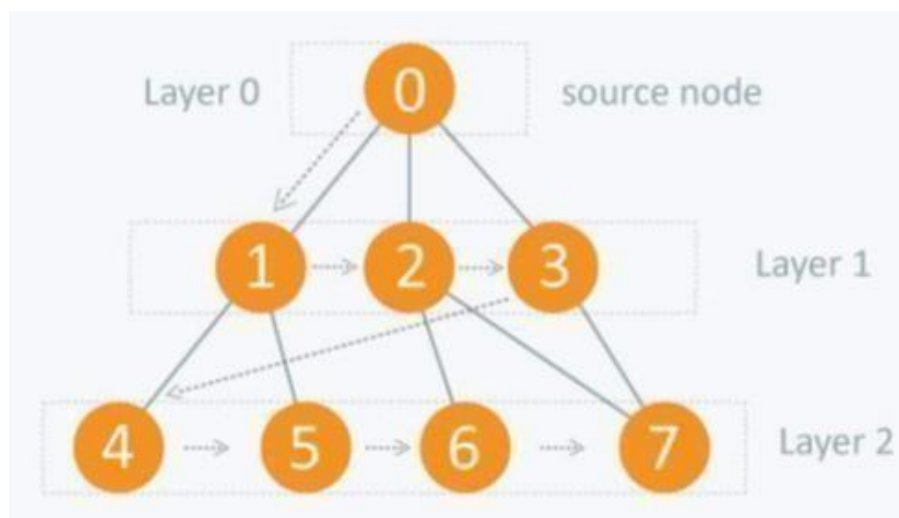
Breadth First Search (BFS)

There are many ways to traverse graphs. BFS is the most commonly used approach. BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layer wise thus exploring the neighbor nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbor nodes.

As the name BFS suggests, you are required to traverse the graph breadthwise as follows:

1. First move horizontally and visit all the nodes of the current layer

Move to the next layer Consider the following diagram.



Parallel Breadth First Search

1. To design and implement parallel breadth first search, you will need to divide the graph into smaller sub-graphs and assign each sub-graph to a different processor or thread.
2. Each processor or thread will then perform a breadth first search on its assigned sub-graph concurrently with the other processors or threads.
3. Two methods: Vertex by Vertex OR Level By Level

Parallel Depth First Search

- Different subtrees can be searched concurrently.
- Subtrees can be very different in size.
- Estimate the size of a subtree rooted at a node.
- Dynamic load balancing is required.

Parameters in Parallel DFS: Work Splitting

- Work is split by splitting the stack into two.
- Ideally, we do not want either of the split pieces to be small.
- Select nodes near the bottom of the stack (node splitting), or
- Select some nodes from each level (stack splitting)
- The second strategy generally yields a more even split of the space.

OpenMP Section Compiler Directive:

A parallel loop is an example of independent work units that are numbered. If you have a pre-determined number of independent work units, the sections is more appropriate. In a sections construct can be any number of section constructs. These need to be independent, and they can be execute by any available thread in the current team, including having multiple sections done by the same thread.

The sections construct is a non-iterative work sharing construct that contains a set of structured blocks that are to be distributed among and executed by the threads in a team. Each structured block is executed once by one of the threads in the team in the context of its implicit task.

Program:

(BFS)

```
from multiprocessing import Process
```

```

import time

graph = {
    0: [1, 2],
    1: [3, 4],
    2: [5],
    3: [],
    4: [],
    5: []
}

visited = set()

def parallel_bfs(start):
    queue = [start]
    visited.add(start)

    while queue:
        node = queue.pop(0)
        print(f"Visiting node: {node} bfs in parallel.")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

        time.sleep(0.3)

if __name__ == "__main__":
    p = Process(target=parallel_bfs, args=(0,))
    p.start()
    p.join()

```

Output:

```
... Visiting node: 0 bfs in parallel.  
    Visiting node: 1 bfs in parallel.  
    Visiting node: 2 bfs in parallel.  
    Visiting node: 3 bfs in parallel.  
    Visiting node: 4 bfs in parallel.  
    Visiting node: 5 bfs in parallel.
```

Program:

(DFS)

```
from multiprocessing import Process #DFS
```

```
import time
```

```
graph = {
```

```
    0: [1, 2],
```

```
    1: [3, 4],
```

```
    2: [5],
```

```
    3: [],
```

```
    4: [],
```

```
    5: []
```

```
}
```

```
visited = set()
```

```
def parallel_dfs(node):
```

```
    visited.add(node)
```

```
    print(f"Visiting node: {node} in parallel.")
```

```
    time.sleep(0.3)
```

```
    processes = []
```

```
    for neighbor in graph[node]:
```

```
        if neighbor not in visited:
```

```
            p = Process(target=parallel_dfs, args=(neighbor,))
```

```
            processes.append(p)
```

```
            p.start()
```

```
    for p in processes:
```

```
        p.join()
```

```
if __name__ == "__main__":
```

```
p = Process(target=parallel_dfs, args=(0,))  
p.start()  
p.join()
```

Output:

```
... Visiting node: 0 in parallel.  
    Visiting node: 1 in parallel.  
    Visiting node: 2 in parallel.  
    Visiting node: 4 in parallel.Visiting node: 3 in parallel.  
  
    Visiting node: 5 in parallel.
```

Conclusion:

Thus, we have successfully implemented parallel algorithms for Depth first search Search and Breadth First Search.

Experiment - 02

Aim:

Write a program to implement Parallel Bubble Sort and Parallel Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Objective:

Student will learn:

1. The Basic Concepts of Bubble Sort and Merge Sort.
2. Multiple Compiler Directives, library routines, environment variables available for OpenMP.

Theory:

Parallel Sorting:

A sequential sorting algorithm may not be efficient enough when we have to sort a huge volume of data. Therefore, parallel algorithms are used in sorting.

Design methodology:

Based on an existing sequential sort algorithm

- Try to utilize all resources available
- Possible to turn a poor sequential algorithm into a reasonable parallel algorithm

Bubble Sort

The idea of bubble sort is to compare two adjacent elements. If they are not in the right order, switch them. Do this comparing and switching (if necessary) until the end of the array is reached. Repeat this process from the beginning of the array n times. Average performance is $O(n^2)$

Bubble Sort Example

Here we want to sort an array containing

[8, 5, 1]. 8, 5, 1 Switch 8 and 5

5, 8, 1 Switch 8 and 1

5, 1, 8 Reached end start again

5, 1, 8 Switch 5 and 1

1, 5, 8 No Switch for 5 and 8

1, 5, 8 Reached end start again.

1, 5, 8 No switch for 1, 5

1, 5, 8 No switch for 5, 8

1, 5, 8 Reached end.

But do not start again since this is the n th iteration of same process

Parallel Bubble Sort

- Implemented as a pipeline.
- Let $\text{local_size} = n / \text{no_proc}$. We divide the array in no_proc parts, and each process executes the bubble sort on its part, including comparing the last element with the first one belonging to the next thread.
- Implement with the loop (instead of $j < i$) for ($j=0; j < n-1; j++$)
- For every iteration of i , each thread needs to wait until the previous thread has finished that iteration before starting.
- We'll coordinate using a barrier.

Algorithm for Parallel Bubble Sort

1. For $k = 0$ to $n-2$
2. If k is even then
3. for $i = 0$ to $(n/2)-1$ do in parallel
4. If $A[2i] > A[2i+1]$ then
5. Exchange $A[2i] \leftrightarrow A[2i+1]$
6. Else
7. for $i = 0$ to $(n/2)-2$ do in parallel
8. If $A[2i+1] > A[2i+2]$ then
9. Exchange $A[2i+1] \leftrightarrow A[2i+2]$
10. Next k

Parallel Bubble Sort

- Implemented as a pipeline.
- Let $\text{local_size} = n / \text{no_proc}$. We divide the array in no_proc parts, and each process executes the bubble sort on its part, including comparing the last element with the first one belonging to the next thread.
- Implement with the loop (instead of $j < i$) for ($j=0; j < n-1; j++$).

- For every iteration of i , each thread needs to wait until the previous thread has finished that iteration before starting.
- We'll coordinate using a barrier.

Algorithm for Parallel Bubble Sort

1. For $k = 0$ to $n-2$
2. If k is even then
3. for $i = 0$ to $(n/2)-1$ do in parallel
4. If $A[2i] > A[2i+1]$ then
5. Exchange $A[2i] \leftrightarrow A[2i+1]$
6. Else
7. for $i = 0$ to $(n/2)-2$ do in parallel
8. If $A[2i+1] > A[2i+2]$ then
9. Exchange $A[2i+1] \leftrightarrow A[2i+2]$
10. Next k

Parallel Bubble Sort Example

- Compare all pairs in the list in parallel.
- Alternate between odd and even phases.
- Shared flag, sorted, initialized to true at beginning of each iteration (2 phases), if any processor perform swap, sorted = false.

Program

```
import time
import random
import threading
start = time.perf_counter()
# Create function to sort elements in odd phase
def odd_phase(lst, n):
    for i in range(0, n-1, 2):
```



```

        if lst[i] > lst[i+1]:
            lst[i], lst[i+1] = lst[i+1], lst[i]
# Create function to sort elements in even phase
def even_phase(lst, n):
    for i in range(1, n-1, 2):
        if lst[i] > lst[i+1]:
            lst[i], lst[i+1] = lst[i+1], lst[i]
# Create function for parallel bubble sort
def Parallel_Bubble_Sort(lst):
    n = len(lst)
    # Create list to keep track of all threads
    threads = []
    for i in range(1, n+1):
        # If i is odd, call for odd_phase function
        if i % 2 == 1:
            # Start threads one by one
            t1 = threading.Thread(target=odd_phase, args=[lst, n])
            t1.start()
            threads.append(t1)
        # If i is even, call for even_phase function
        else:
            t2 = threading.Thread(target=even_phase, args=[lst, n])
            t2.start()
            threads.append(t2)
    for thread in threads:
        thread.join()
    # Print final sorted list
    print(lst)
# Example list to test the program
lst = [random.randint(0, 100) for i in range(100)]
Parallel_Bubble_Sort(lst)
finish = time.perf_counter()
print(f'Finished in {round(finish-start, 2)} second(s)')

```

Output:

(Parallel Bubble Sort)

```
... Sequential Time: 0.048752784729003906
Parallel Time: 0.9326319694519043
Sorted Output (first 10): [17, 42, 47, 48, 58, 73, 77, 108, 115, 153]
```

Merge Sort

- Collects sorted list onto one processor
- Merges elements as they come together
- Simple tree structure
- Parallelism is limited when near the root

Steps of Merge Sort:

To sort $A[p \dots r]$:

1. Divide Step

If a given array A has zero or one element, simply return; it is already sorted. Otherwise, split $A[p \dots r]$ into two subarrays $A[p \dots q]$ and $A[q + 1 \dots r]$, each containing about half of the elements of $A[p \dots r]$. That is, q is the halfway point of $A[p \dots r]$.

2. Conquer Step

Conquer by recursively sorting the two subarrays $A[p \dots q]$ and $A[q + 1 \dots r]$.

3. Combine Step

Combine the elements back in $A[p \dots r]$ by merging the two sorted subarrays $A[p \dots q]$ and $A[q + 1 \dots r]$ into a sorted sequence. To accomplish this step, we will define a procedure $MERGE(A, p, q, r)$.

Example:

Parallel Merge Sort

- Parallelize processing of sub-problems
- Max parallelization achieved with one processor per node (at each layer/height)

Parallel Merge Sort Example

- Perform Merge Sort on the following list of elements. Given 2 processors, P_0 & P_1 .
- 4,3,2,1

Algorithm for Parallel Merge Sort

1. Procedure parallel Merge Sort
2. Begin
3. Create processors P_i where $i = 1$ to n
4. if $i > 0$ then receive size and parent from the root
5. receive the list, size and parent from the root
6. endif
7. $midvalue = listsize/2$
8. if both children is present in the tree then
9. send $midvalue$, first child
10. send $listsize - mid$, second child
11. send list, $midvalue$, first child
12. send list from $midvalue$, $listsize - midvalue$, second child
13. call $mergelist(list, 0, midvalue, list, midvalue + 1, listsize, temp, 0, listsize)$
14. store temp in another array list2
15. else
16. call $parallelMergeSort(list, 0, listsize)$
17. endif
18. if $i > 0$ then
19. send list, listsize, parent
20. endif
21. end

ALGORITHM ANALYSIS

1. Time Complexity Of parallel Merge Sort and parallel Bubble sort in best case is(when alldata is already in sorted form): $O(n)$
2. Time Complexity Of parallel Merge Sort and parallel Bubble sort in worst case is: $O(n \log n)$
3. Time Complexity Of parallel Merge Sort and parallel Bubble sort in average case is: $O(n \log n)$

Program:

```
import time
```

```

def mergeSort(arr):
    if len(arr) > 1:
        # Finding the mid of the array
        mid = len(arr) // 2
        # Dividing the array elements into 2 halves
        L = arr[:mid]
        R = arr[mid:]
        # Sorting the first half
        mergeSort(L)
        # Sorting the second half
        mergeSort(R)
        i = j = k = 0
        # Copy data to temp arrays L[] and R[]
        while i < len(L) and j < len(R):
            if L[i] <= R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1
        # Checking if any element was left
        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1
        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
def printList(arr):
    for i in range(len(arr)):
        print(arr[i], end=" ")

```

```
print()
# Driver Code
if __name__ == '__main__':
    arr = [12, 11, 13, 5, 6, 7]
    print("Given array is", end="\n")
    printList(arr)
    mergeSort(arr)
    print("Sorted array is: ", end="\n")
    printList(arr)
```

Output:

(Parallel Merge Sort)

```
Sequential Time: 0.002452850341796875
Parallel Time: 0.008197546005249023
Sorted Output (first 10): [22, 42, 46, 47, 56, 65, 66, 66, 68, 97]
```

Conclusion:

Thus, we have successfully implemented parallel algorithms for Bubble Sort and Merger Sort.

Experiment -03

Aim:

Implement parallel reduction using Min, Max, Sum and Average Operations.

Objective:

To study and implementation of directive based parallel programming model.

Theory:

OpenMP:

OpenMP is a set of C/C++ pragmas which provide the programmer a high-level front-end interface which get translated as calls to threads. The key phrase here is "higher-level"; the goal is to better enable the programmer to "think parallel" alleviating him/her of the burden and distraction of dealing with setting up and coordinating threads. For example, the OpenMP directive.

OpenMP Core Syntax:

Most of the constructs in OpenMP are compiler directives:

#pragma omp construct [clause [clause]...]

1. The min_reduction function finds the minimum value in the input array using the #pragma omp parallel for reduction (min: min_value) directive, which creates a parallel region and divides the loop iterations among the available threads. Each thread performs the comparison operation in parallel and updates the min_value variable if a smaller value is found.
2. Similarly, the max_reduction function finds the maximum value in the array, sum_reduction function finds the sum of the elements of array and average_reduction function finds the average of the elements of array by dividing the sum by the size of the array.
3. The reduction clause is used to combine the results of multiple threads into a single value, which is then returned by the function. The min and max operators are used for the min_reduction and max_reduction functions, respectively, and the + operator is used for the sum_reduction and average_reduction functions. In the main function, it creates a vector and calls the functions min_reduction, max_reduction, sum_reduction, and average_reduction to compute the values of min, max, sum and average respectively.

Program:

```
import numpy as np
```

```

import multiprocessing as mp

def min_reduction(arr):
    min_value = float('inf')
    for num in arr:
        if num < min_value:
            min_value = num
    return min_value

def max_reduction(arr):
    max_value = float('-inf')
    for num in arr:
        if num > max_value:
            max_value = num
    return max_value

def sum_reduction(arr):
    return sum(arr)

def average_reduction(arr):
    return sum(arr) / len(arr)

def main():
    arr = [5, 2, 9, 1, 7, 6, 8, 3, 4]

    pool = mp.Pool(processes=mp.cpu_count())

    min_value = pool.apply(min_reduction, args=(arr,))
    max_value = pool.apply(max_reduction, args=(arr,))
    sum_value = pool.apply(sum_reduction, args=(arr,))
    average_value = pool.apply(average_reduction, args=(arr,))

    print(f"Minimum value: {min_value}")
    print(f"Maximum value: {max_value}")
    print(f"Sum: {sum_value}")
    print(f"Average: {average_value}")

if __name__ == "__main__":
    main()

```

OUTPUT:

```
... OUTPUT:  
  
Minimum value: 2  
Maximum value: 18  
Sum: 90  
Average: 10.0
```

Conclusion:

Thus, we have successfully implemented parallel reduction using Min, Max, Sum and Average Operations.

Experiment – 04

Aim:

To write a CUDA program for, find.

1. Addition of two large vectors.
2. Matrix Multiplication using CUDA C.

Objective:

To study and implement the CUDA program using vectors.

Theory:

CUDA:

CUDA programming is especially well-suited to address problems that can be expressed as data- parallel computations. Any applications that process large data sets can use a data- parallel model to speed up the computations. Data-parallel processing maps data elements to parallel threads.

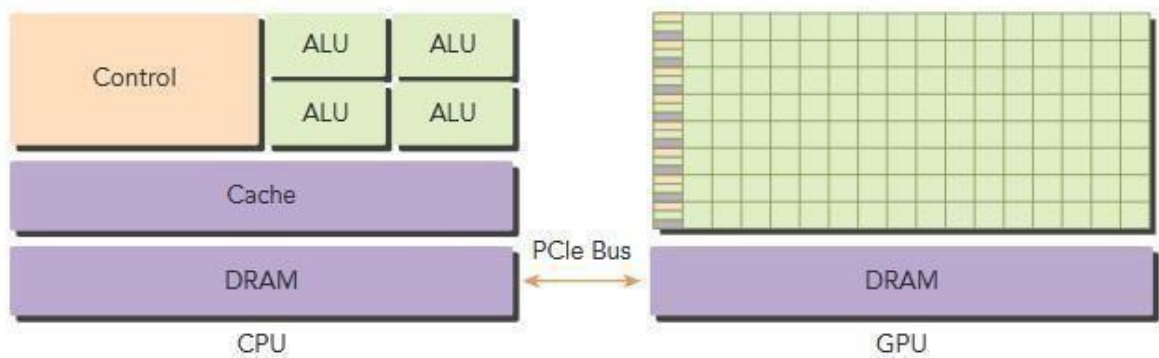
The first step in designing a data parallel program is to partition data across threads, with each thread working on a portion of the data. The first step in designing a data parallel program is to partition data across threads, with each thread working on a portion of the data.

CUDA Architecture:

A heterogeneous application consists of two parts:

- Host code
- Device code

Host code runs on CPUs and device code runs on GPUs. An application executing on a heterogeneous platform is typically initialized by the CPU. The CPU code is responsible for managing the environment, code, and data for the device before loading compute-intensive tasks on the device. With computational intensive applications, program sections often exhibit a rich amount of data parallelism. GPUs are used to accelerate the execution of this portion of data parallelism. When a hardware component that is physically separate from the CPU is used to accelerate computationally intensive sections of an application, it is referred to as a hardware accelerator. GPUs are arguably the most common example of a hardware accelerator. GPUs must operate in conjunction with a CPU-based host through PCI-Express bus, as shown in Figure.



Matrix-Matrix Multiplication

- Consider two $n \times n$ matrices A and B partitioned into p blocks $A_{i,j}$ and $B_{i,j}$ ($0 \leq i < j$) of size each. $(n/\sqrt{p}) \times (n/\sqrt{p})$
- Process $P_{i,j}$ initially stores $A_{i,j}$ and $B_{i,j}$ and computes block $C_{i,j}$ of the result matrix.
- Computing submatrix $C_{i,j}$ requires all submatrices $A_{i,k}$ and $B_{k,j}$ for $0 \leq k < \sqrt{p}$
- All-to-all broadcast blocks of A along rows and B along columns.

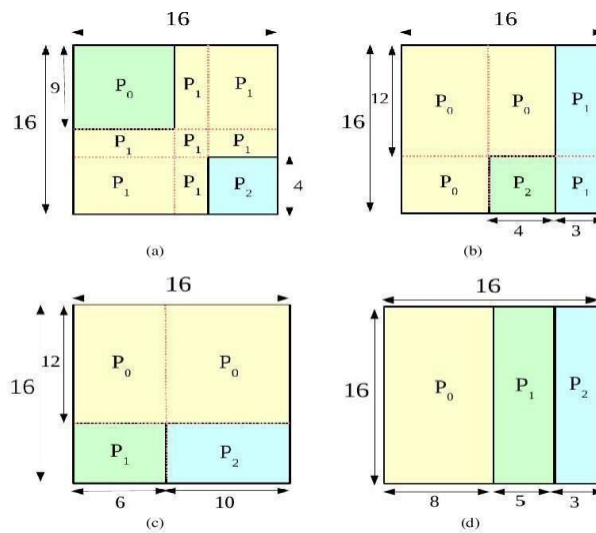


Figure 1: Different partition shapes for three processors $\{P_0, P_1, P_2\}$ solving PMM of matrices A, B, C of size 16×16

- Perform local submatrix multiplication

Program:

```
#include <iostream>
#define N 10
void add(int *a, int *b, int *c)
{
    for (int tid = 0; tid < N; tid++)
    {
        c[tid] = a[tid] + b[tid];
    }
}
```

```

    }
}

int main(void)
{
    int a[N], b[N], c[N];

    // Fill the arrays 'a' and 'b'
    for (int i = 0; i < N; i++)
    {
        a[i] = -i;
        b[i] = i * i;
    }

    add(a, b, c);

    // Display the results
    for (int i = 0; i < N; i++)
    {
        std::cout << a[i] << " + " << b[i] << " = " << c[i] << std::endl;
    }

    return 0;
}

```

OUTPUT:

```

...  0 + 0 = 0
      -1 + 1 = 0
      -2 + 4 = 2
      -3 + 9 = 6
      -4 + 16 = 12
      -5 + 25 = 20
      -6 + 36 = 30
      -7 + 49 = 42
      -8 + 64 = 56
      -9 + 81 = 72

```

Program:

(Matrix Multiplication)

```
import numpy as np
```

```
N = 3
```

```
A = np.random.randint(1, 5, (N, N))
```

```
B = np.random.randint(1, 5, (N, N))
```

```
C = np.dot(A, B)
```

```
print("Matrix A:\n", A)
```

```
print("\nMatrix B:\n", B)
```

```
print("\nResult Matrix:\n", C)
```

OUTPUT:

```
... Matrix A:
  [[2 2 3]
  [3 4 4]
  [1 2 4]]

  Matrix B:
  [[2 2 4]
  [3 3 2]
  [2 3 1]]

  Result Matrix:
  [[16 19 15]
  [26 30 24]
  [16 20 12]]
```

Conclusion:

Thus, we have successfully implemented the Addition of two large vectors and Matrix Multiplication using CUDA C Programming.